Natural Language-Based Insider Threat Detection from Enterprise CommunicationsA CAPSTONE PROJECT REPORT

Submitted in partial fulfilment of the requirements

for the Course of

ECA4711 – Principles of Digital System Design

to the award of the degree of

BACHELOR OF TECHNOLOGY

IN

ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

Submitted by

Mithlesh N (192424364)

Under the Supervision of

Dr. Kumaragurubaran T

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai–602105

SEPTEMBER 2026

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai–602105

BONAFIDE CERTIFICATE

This is to certify that the Capstone Project entitled "**Natural Language-Based Insider Threat Detection from Enterprise Communications"** has been carried out by **Abinesh H(192424387)**, **Mithlesh N (192424364),** and **Devin Pakianath (192325003)** under the supervision of **Dr. Kumaragurubaran T** and is submitted in partial fulfilment of the requirements for the current semester of the **B. Tech Artificial Intelligence and Data Science** program at Saveetha Institute of Medical and Technical Sciences, Chennai.

COURSE COORDINATOR	COURSE FACULTY / GUIDE
Dr. Sriramya	Dr. Kumaragurubaran T
Program Director	Professor
Department of CSE	Department of CSE
SIMATS Engineering	SIMATS Engineering

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai–602105

DECLARATION

We, **Abinesh H (192424387)**, **Mithlesh N (192424364)** and **Devin Pakianath (192325003)** of the Department of **B. Tech Artificial Intelligence and Data Science (AI&DS)**, SIMATS Engineering, Saveetha Institute of Medical and Technical Sciences, Chennai, hereby declare that the Capstone Project Work entitled "**Natural Language-Based Insider Threat Detection from Enterprise Communications**" is the result of our own Bonafide efforts .

To the best of our knowledge, the work presented herein is original, accurate, and has been carried out in accordance with the principles of engineering ethics and academic integrity. All sources, references, data, and other materials used in the project have been appropriately acknowledged.

We further declare that the data, results, analysis, and findings presented in this report have not been fabricated, falsified, or manipulated. Any external tools, software, datasets, computational resources, or AI-assisted tools used during the project have been appropriately disclosed. We confirm that all applicable ethical, academic, institutional, and professional requirements have been duly followed throughout the planning, execution, analysis, and documentation of the project.

Place: Chennai

Date: 05/09/2026

Signature of the Students with Names

Mithlesh N (192424364)

INDIVIDUAL CONTRIBUTION STATEMENT

(Mandatory for all team projects)

Student Name / Register No.	Specific Responsibilities	Design & Development Contribution	Testing & Analysis Contribution	Report Contribution	Approx. Contribution (%)
Abinesh H (192424387)	Configured the LDR sensor to detect ambient light intensity.	Designed the LDR interfacing circuit with Arduino Uno for light-level detection.	Tested LDR readings under different lighting conditions and verified sensor accuracy.	Documented the LDR working principle, circuit connections, and sensor results.	**34%**
Mithlesh N (192424364)	Developed the control logic for automatic street light operation.	Implemented Arduino-based switching of the LED street light using a transistor.	Tested automatic ON/OFF operation under day and night conditions.	Documented the control algorithm, Arduino program, and system performance.	**33%**
Devin Pakianath (192325003)	Developed the security monitoring dashboard, alert management, and threat investigation features.	Designed and imp the React/Vite da interface and inte with the backend	Tested dashboard functionality, alert generation, risk visualization, and investigation workflows.	Prepared the Module 3 implementation, testing, results, and documentation sections.	**33%**
TOTAL					**100%**

ABSTRACT

The increasing use of enterprise communication platforms such as emails, instant messaging, and internal communication systems has created new challenges in identifying insider threats. Malicious or compromised insiders may misuse organizational information by sharing confidential data, communicating suspicious content, or performing activities that deviate from normal communication patterns. Traditional security systems mainly depend on predefined rules and signatures, which may not effectively identify threats expressed through natural language. Therefore, there is a need for an intelligent system capable of analyzing enterprise communications and identifying potential insider threats at an early stage.

This project proposes a **Natural Language-Based Insider Threat Detection System** that applies Natural Language Processing (NLP) and machine learning techniques to enterprise communication data. The system processes communication text through cleaning, normalization, tokenization, and feature extraction to identify linguistic and behavioral characteristics. The extracted features are analyzed using a threat detection and risk assessment module to classify communications based on their potential threat level and generate an appropriate risk score. A security monitoring dashboard is then used to display detected threats, risk levels, alerts, and investigation information for security personnel.

The proposed methodology integrates communication preprocessing, NLP-based feature extraction, machine learning-based threat classification, risk scoring, alert generation, and interactive security monitoring. The system provides faster identification of suspicious communications and supports systematic investigation of potential insider activities. The results demonstrate that combining linguistic analysis with risk assessment can improve the visibility and prioritization of potential insider threats.

The project concludes that NLP-based analysis can provide an effective additional layer of enterprise security by enabling organizations to detect suspicious communication patterns and respond to potential insider threats more efficiently.

Keywords: Natural Language Processing, Insider Threat Detection, Enterprise Security, Machine Learning, Risk Assessment, Security Monitoring

TABLE OF CONTENTS

LIST OF FIGURES

LIST OF TABLES

CHAPTER 1
INTRODUCTION AND ENGINEERING PROBLEM

1.1 Background

The rapid adoption of digital communication systems has transformed the way organizations exchange information. Enterprise environments commonly use emails, instant messaging platforms, collaboration tools, and internal communication systems for sharing operational, financial, technical, and confidential information. Although these systems improve productivity, they also introduce security risks associated with insider activities.

An **insider threat** occurs when an authorized user intentionally or unintentionally performs an activity that compromises the confidentiality, integrity, or availability of organizational information. Unlike conventional external attacks, insider threats can be difficult to identify because legitimate users already have access to organizational resources. Suspicious behavior may also appear as normal communication when examined using conventional security mechanisms.

Natural Language Processing (NLP) provides techniques for extracting meaningful information from textual communications. By applying text preprocessing, linguistic feature extraction, sentiment and contextual analysis, and machine learning-based classification, suspicious communication patterns can be identified. However, communication-based threat detection requires the integration of multiple components, including data processing, feature extraction, threat classification, risk assessment, alert generation, and security investigation.

This project focuses on developing a **Natural Language-Based Insider Threat Detection System from Enterprise Communications**. The proposed system consists of three major modules: **Enterprise Communication Processing & Feature Extraction**, **Insider Threat Detection & Risk Assessment**, and **Security Monitoring Dashboard, Alert Management & Threat Investigation**. The system is intended to provide security personnel with an automated mechanism for identifying and prioritizing potentially suspicious communications.

1.2 Need for the Project

Why the Problem Needs to Be Addressed

Enterprise communications may contain sensitive information such as credentials, confidential documents, financial details, project information, customer data, and internal organizational discussions. Malicious insiders or compromised accounts

may exploit legitimate access to communicate or disclose sensitive information.

Manual inspection of large volumes of communication is time-consuming and difficult to scale. Therefore, an automated system is required to process large amounts of textual communication and identify potentially suspicious content efficiently.

Who Is Affected by the Problem

The problem primarily affects:

- Organizations handling confidential or sensitive information.
- Security Operations Center (SOC) teams.
- System and network administrators.
- Data protection and compliance teams.
- Employees whose accounts may be compromised.
- Customers and business partners whose information is stored by organizations.

Existing Limitations

Conventional security approaches have several limitations:

1. Rule-based systems may fail to detect threats expressed using different language patterns.
2. Large communication volumes make continuous manual monitoring impractical.
3. Traditional systems may generate excessive alerts without effective prioritization.
4. Textual context and linguistic characteristics are often not sufficiently considered.
5. Security teams require better visualization and investigation support for detected threats.
6. A single security indicator may not accurately represent the overall risk associated with a communication.

Engineering Significance

The project has engineering significance because it integrates **NLP, machine learning, risk assessment, data processing, and security monitoring** into a unified system. The solution addresses practical constraints involving data volume, classification accuracy, processing time, false positives, and usability. It also demonstrates how intelligent software engineering techniques can be applied to

strengthen enterprise cybersecurity.

1.3 Problem Statement

Enterprise organizations generate large volumes of textual communications through emails, messaging systems, and internal collaboration platforms, making manual identification of malicious, suspicious, or abnormal communication impractical.

The engineering problem is to **design and implement an intelligent system that can process enterprise communication text, extract relevant linguistic and contextual features, identify potential insider-threat indicators, estimate associated risk levels, and present prioritized alerts for security investigation**.

The problem involves multiple interacting factors, including:

- Natural-language variability and contextual ambiguity.
- Different communication patterns among legitimate users.
- Classification accuracy and false-positive reduction.
- Threat severity and risk prioritization.
- Large-scale communication processing.
- Timely alert generation.
- Usability of security monitoring interfaces.
- Privacy and controlled handling of enterprise communication data.

The proposed solution must balance **detection effectiveness, processing efficiency, interpretability, scalability, and security constraints** while avoiding unnecessary alerts for legitimate communications.

1.4 Project Aim

The overall aim of this project is to **design and develop an NLP and machine learning-based system for detecting potential insider threats from enterprise communications, assessing their risk levels, and supporting security monitoring and threat investigation through an interactive dashboard.**

1.5 Project Objectives

The project objectives are:

1. **To design** a system architecture for processing and analyzing enterprise communication data.
2. **To develop** a communication preprocessing pipeline for cleaning, normalizing, and preparing textual data for analysis.
3. **To extract** relevant linguistic, contextual, and communication-based features using Natural Language Processing techniques.

4. **To implement** a machine learning-based mechanism for identifying potentially suspicious or threat-related communications.
5. **To develop** a risk assessment mechanism for assigning appropriate risk levels to detected threats.
6. **To implement** an alert management system for recording, prioritizing, and tracking detected security alerts.
7. **To develop** a security monitoring dashboard for visualizing communication analysis, threat levels, risk scores, and alerts.
8. **To provide** investigation capabilities that assist security personnel in examining suspicious communications and associated threat information.
9. **To test and evaluate** the system using appropriate performance measures such as classification accuracy, precision, recall, F1-score, and processing performance.

1.6 Scope

Included

The scope of the project includes:

- Processing textual enterprise communications.
- Text cleaning and normalization.
- Tokenization and other NLP preprocessing techniques.
- Extraction of relevant textual and contextual features.
- Identification of suspicious or threat-related communications.
- Machine learning-based threat classification.
- Risk-score generation and threat-level categorization.
- Alert creation and management.
- Dashboard-based security monitoring.
- Threat investigation and visualization.
- Evaluation of the detection system using suitable performance metrics.

Excluded

The following are outside the primary scope of the project:

- Direct monitoring of physical employee activities.
- Network packet-level intrusion detection.
- Malware reverse engineering.
- Endpoint hardware monitoring.
- Automatic disciplinary action against employees.

- Direct integration with production enterprise communication systems.
- Autonomous blocking or deletion of user communications.
- Real-time surveillance of personal communications outside the authorized enterprise environment.

Assumptions

The project assumes that:

- Communication data is available in a suitable textual format.
- The data used for development and testing is legally and ethically obtained.
- Appropriate labels or threat categories are available for model training and evaluation.
- The system operates within an authorized enterprise security environment.
- Security personnel are responsible for reviewing and acting upon generated alerts.

Boundary Conditions

The system operates primarily on **textual communication content and associated analytical information**. Detection performance depends on the quality, diversity, and representativeness of the available communication dataset. The system provides decision support and risk prioritization rather than independently determining whether an individual is malicious.

1.7 Expected Engineering Outcome

The expected outcome is a functional **prototype of an NLP-based Insider Threat Detection and Security Monitoring System** consisting of three integrated modules:

Module 1 – Enterprise Communication Processing & Feature Extraction

This module will accept enterprise communication data and perform preprocessing, text normalization, NLP analysis, and feature extraction. The output will be structured information suitable for threat analysis.

Module 2 – Insider Threat Detection & Risk Assessment

This module will analyze the extracted features using machine learning techniques to identify potentially suspicious communications. It will assign threat classifications and calculate risk levels to prioritize potentially significant cases.

Module 3 – Security Monitoring Dashboard, Alert Management & Threat Investigation

This module will provide an interactive dashboard through which security personnel can monitor detected threats, view risk scores, manage alerts, and investigate

suspicious communications.

Overall, the project is expected to produce a **working software prototype and integrated analytical process** capable of transforming raw enterprise communications into structured security intelligence. The system will demonstrate how NLP and machine learning can support earlier identification, prioritization, and investigation of potential insider threats while maintaining appropriate engineering and security constraints.

CHAPTER 2
LITERATURE REVIEW AND EXISTING SOLUTIONS

2.1 Review Method

A structured literature review was carried out to identify existing research, technologies, datasets, security guidelines, and detection approaches relevant to **Natural Language-Based Insider Threat Detection from Enterprise Communications**. The review focused on research published in cybersecurity, machine learning, deep learning, and Natural Language Processing (NLP).

Relevant literature was identified using academic and technical sources including **IEEE Xplore, ScienceDirect, Springer, ACM, NIST, CISA, and Carnegie Mellon University Software Engineering Institute resources**. Search terms included *insider threat detection*, *NLP for cybersecurity*, *machine learning insider threat detection*, *enterprise communication analysis*, *user behavior anomaly detection*, and *insider risk assessment*.

Recent review studies show that insider-threat research has increasingly moved from conventional rule-based approaches toward machine learning, deep learning, NLP, and hybrid approaches. A 2025 systematic review specifically examining NLP-based insider-threat detection analyzed 66 high-quality studies from literature published between 2019 and 2024.

In addition to research papers, established security guidance was reviewed to understand practical insider-threat detection and risk-management requirements. CISA identifies both behavioral and technical indicators that organizations can use

when developing insider-threat detection capabilities. NIST SP 800-53 also recommends an insider-threat program involving centralized integration and analysis of technical and nontechnical information.

2.2 Review of Existing Work

2.2.1 Traditional Rule-Based and Behavioral Approaches

Traditional insider-threat detection systems commonly rely on predefined rules, thresholds, signatures, and manually selected behavioral indicators. These systems monitor activities such as unusual login times, excessive file access, unauthorized data transfers, or abnormal system usage.

CISA's Insider Threat Mitigation Guide identifies a broad range of behavioral and technical indicators, including repeated policy violations, unexplained access to sensitive systems, unusual working hours, and excessive data-copying activities.

Advantages:

- Simple to implement.
- Easy to understand and audit.
- Effective for known and clearly defined behaviors.
- Low computational complexity.

Limitations:

- Difficult to capture previously unknown threat patterns.
- Highly dependent on manually defined rules.
- Can produce false positives when legitimate users behave unusually.
- Limited ability to understand the meaning and context of natural-language communication.

These limitations create the need for adaptive, data-driven approaches.

2.2.2 Machine Learning-Based Insider Threat Detection

Machine learning has been widely investigated for identifying anomalous insider behavior. Techniques such as decision trees, random forests, support vector machines, clustering, anomaly detection, and ensemble methods can learn patterns from historical user activity.

Research using the CERT Insider Threat Dataset has modeled user activity using features derived from logon, email, HTTP, file, and device activity. Such approaches demonstrate that combining multiple activity sources can improve the identification of abnormal user behavior.

The CERT dataset is particularly important in insider-threat research because it

provides synthetic background activity together with simulated malicious-user scenarios.

Advantages:

- Can identify patterns that are difficult to express as fixed rules.
- Supports automated classification.
- Can combine multiple behavioral features.
- Suitable for large datasets.

Limitations:

- Performance depends heavily on the quality and representativeness of training data.
- Insider-threat datasets are often synthetic or imbalanced.
- Traditional feature engineering may fail to capture complex communication context.
- Some models provide limited explanations for their predictions.

A recent IEEE review highlights data imbalance, heterogeneous features, and interpretability as continuing challenges in machine-learning-based insider-threat detection.

2.2.3 Deep Learning-Based Approaches

Deep learning has been introduced to overcome some limitations of conventional machine learning. Neural networks can learn complex relationships from high-dimensional and heterogeneous data, reducing the dependence on manually designed features.

A review of deep learning for insider-threat detection notes that conventional machine-learning approaches can struggle with high-dimensionality, complexity, heterogeneity, sparsity, limited labeled data, and the subtle nature of insider threats. Deep learning can provide an end-to-end learning approach, although the lack of labeled data and adaptive attacks remain significant challenges.

Advantages:

- Learns complex patterns automatically.
- Can process high-dimensional data.
- Suitable for sequential and contextual analysis.
- Can be extended to text and multimodal information.

Limitations:

- Requires substantial training data and computational resources.

- More difficult to interpret.
- May overfit limited datasets.
- Deployment can be more complex than traditional machine-learning models.

2.2.4 NLP-Based Insider Threat Detection

Natural Language Processing provides an important approach for analyzing the textual content of enterprise communications. Instead of treating an email or message only as an event, NLP allows the actual language to be analyzed for potentially suspicious content, intent, contextual relationships, and linguistic patterns.

Recent research has identified NLP and large language model approaches as promising techniques for detecting malicious insider activity because they can extract information from textual and contextual content.

A 2025 systematic review specifically focused on NLP-based insider-threat detection and found that research increasingly uses combinations of NLP, machine learning, and deep learning. However, the review also identified limitations involving synthetic datasets, explainability, binary classification bias, model complexity, and insufficient real-time analysis.

Advantages:

- Directly analyzes communication content.
- Captures linguistic and contextual information.
- Can identify suspicious language patterns.
- Can complement conventional behavioral indicators.
- Suitable for communication-focused threat detection.

Limitations:

- Natural language is ambiguous and context dependent.
- Sarcasm, indirect language, and domain-specific terminology can be difficult to interpret.
- Privacy and ethical concerns must be considered.
- High-quality labeled enterprise communication datasets are difficult to obtain.

2.2.5 CERT Insider Threat Dataset

The **CERT Insider Threat Test Dataset**, developed through Carnegie Mellon University's CERT Division and partners, is one of the most widely used benchmark datasets for insider-threat research. It contains synthetic background activity and

malicious-actor scenarios.

The dataset contains different types of enterprise activity, including email, logon, HTTP, file, and device information. This makes it useful for developing and evaluating machine-learning-based insider-threat detection systems.

However, the dataset is synthetic rather than actual enterprise communication data. Consequently, models trained on it may not fully represent the linguistic diversity, organizational culture, and communication patterns of real organizations.

2.2.6 Security Standards and Guidelines

Security standards provide an operational framework for insider-threat management. NIST SP 800-53 includes an **Insider Threat Program** control that emphasizes the integration and analysis of technical and nontechnical information to identify potential insider-threat concerns.

CISA similarly emphasizes detecting, identifying, assessing, and managing insider threats as interconnected activities rather than treating detection as an isolated process.

2.3 Comparative Analysis

Existing Approach	Advantages	Limitations	Relevance to Proposed Work
Rule-Based Detection	Simple, interpretable, easy to implement	Static rules, poor adaptability, false positives	Provides baseline threat indicators and rule concepts
User Behavior Anomaly Detection	Detects abnormal activity patterns	May not understand communication content	Supports behavioral risk assessment
Traditional Machine Learning	Automated classification, efficient, suitable for structured data	Depends on feature engineering and training data	Used for threat classification and risk prediction
Deep Learning	Learns complex patterns and high-dimensional relationships	Computationally expensive and less interpretable	Provides advanced learning possibilities
NLP-Based Detection	Understands textual and linguistic characteristics	Context ambiguity and dataset limitations	Core technology for communication analysis
CERT Dataset-Based Systems	Standardized benchmark and labeled threat scenarios	Synthetic data may not represent real organizations	Useful for model development and evaluation
Security Standards /	Provide structured security practices	Do not provide a complete automated	Guides risk assessment, monitoring, and

Guidelines		detection system	investigation
Proposed System	Integrates NLP, ML, risk assessment, alerts, dashboard, and investigation	Dependent on dataset quality and model performance	Provides an integrated communication-focused insider-threat prototype

2.4 Research / Engineering Gap

The literature review identifies several unresolved research and engineering challenges.

1. Limited Integration of Communication Content and Risk Assessment

Many existing systems concentrate on user activity, system logs, or behavioral anomalies. Although NLP-based approaches increasingly analyze textual communication, there remains a need for systems that combine **communication-level NLP analysis with explicit threat-risk assessment and security alerting**.

2. Dependence on Synthetic Datasets

The CERT dataset is widely used because real enterprise insider-threat data is difficult to obtain. However, it is synthetic, which limits direct generalization to real organizational environments.

3. High False-Positive Risk

Legitimate employees may discuss sensitive subjects, work under unusual conditions, or use terminology that resembles suspicious communication. Therefore, simply identifying keywords is insufficient. Contextual and combined feature analysis is required.

4. Explainability

Advanced machine-learning and deep-learning models may achieve strong predictive performance but can make it difficult for security analysts to understand why a communication was classified as suspicious. Recent literature identifies interpretability as an important unresolved issue.

5. Real-Time and Operational Monitoring

Research often focuses on model performance using offline datasets. There is comparatively less emphasis on integrating detection models with practical dashboards, alert management, and investigation workflows. The 2025 systematic review specifically identifies real-time analysis as an area requiring further development.

6. Imbalanced Threat Data

Insider-threat events are relatively rare compared with legitimate activities. This creates a class-imbalance problem that can cause a model to favor normal communications and miss important threats. Recent surveys identify data imbalance as a continuing challenge.

7. Balancing Security and Privacy

Enterprise communication analysis can involve sensitive information. Therefore, an engineering solution must consider controlled access, appropriate datasets, data minimization, and responsible handling of communication content.

2.5 Proposed Contribution

The proposed project addresses the identified gaps by developing an integrated **Natural Language-Based Insider Threat Detection System from Enterprise Communications**.

The major contribution is the integration of three complementary modules:

Module 1 – Enterprise Communication Processing & Feature Extraction

The system processes enterprise communication text through NLP preprocessing and feature extraction. Relevant textual and contextual characteristics are converted into machine-readable features for subsequent analysis.

Module 2 – Insider Threat Detection & Risk Assessment

The extracted features are analyzed using machine-learning techniques to identify potentially suspicious communications. Instead of producing only a binary classification, the system incorporates **risk assessment and threat-level prioritization**, allowing security personnel to focus on higher-risk cases.

Module 3 – Security Monitoring Dashboard, Alert Management & Threat Investigation

The detected threats are presented through an integrated dashboard. Security personnel can view threat classifications, risk scores, alerts, and investigation-related information from a centralized interface.

CHAPTER 3

ENGINEERING DESIGN & TRADE-OFFS, SUSTAINABILITY, ETHICS, SAFETY, RISK &

SECURITY

3.1 Requirements

3.1.1 Stakeholder / User Requirements

Stakeholder / User	Requirement
Security Analyst	Identify potentially suspicious enterprise communications and view their associated risk levels.
Security Administrator	Monitor alerts, manage threat information, and review system activity from a centralized dashboard.
Incident Investigator	Access relevant communication details, threat indicators, risk scores, and investigation information.
Organization / Management	Obtain an overview of insider-threat trends and high-risk activities for security decision-making.
System Developer	Maintain a modular, testable, and maintainable NLP and machine-learning system.
Data/Privacy Officer	Ensure communication data is handled securely and only used for authorized security purposes.
End User	Ensure that legitimate communications are not unnecessarily classified as threats.

3.1.2 Functional & Non-Functional Requirements

Requirement Type	Requirement	Measurable Target
Functional	Accept enterprise communication text as input	100% of valid test inputs processed
Functional	Preprocess communication data	Cleaning, normalization and tokenization completed for every valid input
Functional	Extract NLP features	Generate feature representation for every processed communication
Functional	Classify communication	Produce a threat classification for each analyzed communication
Functional	Calculate risk	Generate a numerical risk score and corresponding threat level
Functional	Generate alerts	Create alerts for communications exceeding the configured risk threshold
Functional	Display security	Show threats, risk scores, alerts and

	information	statistics on the dashboard
Functional	Support investigation	Allow authorized users to inspect detected threat information
Functional	Store analysis results	Maintain structured records of analyzed communications and alerts
Non-Functional	Performance	Process an individual communication within an acceptable response time under test conditions
Non-Functional	Accuracy	Achieve a target F1-score of at least 0.80 on the selected evaluation dataset
Non-Functional	Reliability	Successfully process at least 95% of valid test requests without application failure
Non-Functional	Security	Implement authentication and role-based access to protected system functions

3.2 Constraints & Applicable Standards

The project primarily involves **software, enterprise communication data, cybersecurity, NLP, and machine learning**. Therefore, the following constraints and standards are directly applicable.

Standard / Code	Requirement	How Applied in This Project
ISO/IEC 27001:2022 – Information Security Management Systems	Establish controls for confidentiality, integrity, availability and information-security risk management.	Used as a security-management reference for protecting communication data, authentication, access control and risk handling.
ISO/IEC 27002:2022 – Information Security Controls	Provides guidance for implementing information-security controls.	Used to guide access control, information protection, logging, monitoring and secure handling of sensitive data.
ISO/IEC 23894:2023 – Information technology — Artificial intelligence — Guidance on risk management	Provides guidance for identifying and managing AI-related risks.	Used to consider model errors, false positives, false negatives, data quality and limitations of the detection model.
ISO/IEC 42001:2023 – AI Management System	Provides requirements for responsible	Used as a reference for responsible AI development,

	management of AI systems.	transparency, risk management and continual evaluation.
OWASP Application Security Verification Standard (ASVS)	Provides requirements for web application security verification.	Relevant to authentication, authorization, session management, input validation and secure application development for the monitoring dashboard and backend API.

Applicable Engineering Constraints

- **Data constraint:** Real enterprise communications are difficult to obtain because of confidentiality and privacy requirements.
- **Dataset constraint:** Training and evaluation may rely on synthetic, anonymized, or publicly available datasets.
- **Computational constraint:** NLP and machine-learning models must operate within the computational resources available for the prototype.
- **Security constraint:** Only authenticated and authorized users should access threat information.
- **Accuracy constraint:** False positives can unnecessarily burden security analysts, while false negatives can allow genuine threats to remain undetected.
- **Ethical constraint:** A model prediction must not be treated as definitive proof that an employee is malicious.
- **Integration constraint:** The three project modules must communicate through well-defined interfaces and structured data formats.

3.3 Design Specifications

Parameter	Target / Requirement	Unit	Priority
Input type	Enterprise communication text	Text	High
Text preprocessing	Cleaning, normalization and tokenization	% valid inputs processed	High
Feature extraction	NLP-based feature representation	Features/vector	High
Threat classification	Normal/suspicious or defined threat classes	Class	High
Risk score	Normalized threat-risk value	0–100	High
Risk categories	Low, Medium, High, Critical	Level	High
Detection	Target F1-score ≥ 0.80	Score	High

performance			
Alert generation	Alert for configured high-risk threshold	Boolean	High
Dashboard response	Responsive display of analytical results	Seconds	Medium
Authentication	Authorized-user authentication	Boolean	High
Access control	Role-based access to protected functions	Boolean	High
Data storage	Structured storage of analysis and alert information	Records	High

3.4 Alternative Solutions & Evaluation

Alternative 1: Rule-Based Detection

A rule-based system identifies suspicious communications using predefined keywords, patterns, thresholds, and manually configured security rules. It is straightforward to implement and provides highly interpretable decisions.

Alternative 2: Traditional Machine Learning

A traditional machine-learning approach extracts features from communications and uses models such as Logistic Regression, Support Vector Machine, Random Forest, or similar classifiers to identify potential threats. This approach provides a balance between computational efficiency, classification capability, and implementation complexity.

Alternative 3: NLP + Machine Learning Integrated System

The proposed approach combines NLP preprocessing and feature extraction with machine-learning-based threat classification, risk scoring, alert management, and a security dashboard. It provides an end-to-end workflow from communication analysis to threat investigation.

Evaluation Matrix

Scoring: 1 = Poor, 3 = Moderate, 5 = Excellent.

Criterion	Weight	Alternative 1: Rule-Based	Alternative 2: ML	Alternative 3: NLP + ML Integrated
Performance	30%	2	4	5
Cost	15%	5	4	3
Safety	15%	3	4	4
Sustainability	15%	5	4	4

Reliability	25%	3	4	5
Weighted Score	**100%**	**3.35/5**	**4.00/5**	**4.35/5**

Interpretation

The evaluation matrix gives the **NLP + ML integrated system the highest weighted score of 4.35/5**, compared with 4.00/5 for traditional machine learning and 3.35/5 for rule-based detection. Therefore, the integrated approach provides the strongest overall balance for the requirements of this project.

3.5 Selected Design & Detailed Design

3.5.1 Final Design Selection

Alternative 3: NLP + Machine Learning Integrated System is selected because it achieves the highest weighted evaluation score.

The selected design provides better integration between communication analysis, feature extraction, threat classification, risk assessment, alert management, and investigation. Unlike a purely rule-based system, it can learn patterns from data, while the inclusion of risk scoring and dashboard functions makes the solution more suitable for practical security monitoring.

3.5.2 System Architecture Diagram

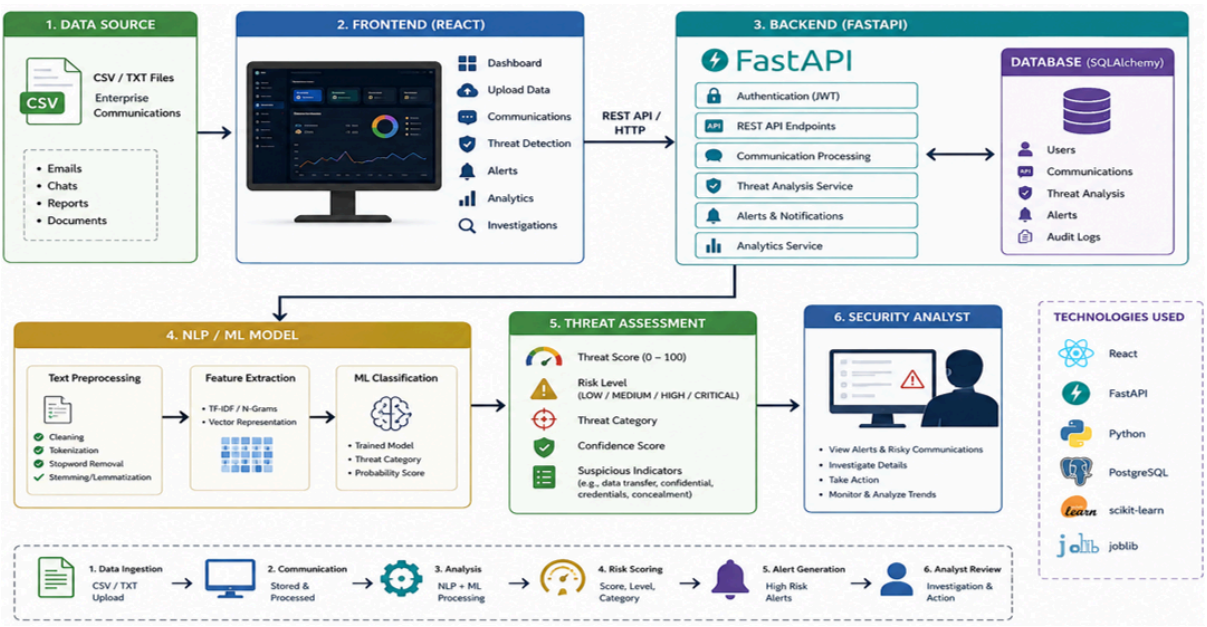


Figure 3.1: Proposed System Architecture

3.5.3 Module Interface Design

The system is designed as three dependent but modular subsystems.

Module 1 receives raw communication and produces structured NLP features. These features are transferred to **Module 2**, which performs threat classification and risk assessment. The resulting threat information is stored and forwarded to **Module 3**, where security personnel can monitor alerts and investigate suspicious communications.

The dashboard does not directly perform the machine-learning analysis; instead, it obtains results through the backend/API layer. This separation improves maintainability and allows the detection model to be modified without redesigning the complete user interface.

3.5.4 Essential Engineering Calculations

Risk Score

A normalized risk score can be calculated using weighted detection factors:

$$R = w_1C + w_2B + w_3L + w_4H$$

Where:

- R= overall risk score
- C= content/threat score
- B= behavioral/context score
- L= linguistic feature score
- H= historical or contextual risk score
- w_1,w_2,w_3,w_4= assigned weights

The weights are selected such that:

$$w_1 + w_2 + w_3 + w_4 = 1$$

The final score is normalized to a **0–100 scale**.

A prototype risk classification can be defined as:

Risk Score	Threat Level
0–24	Low
25–49	Medium
50–74	High
75–100	Critical

These thresholds can be tuned experimentally based on validation results.

Classification Metrics

The system can be evaluated using:

$$\text{Precision} = \frac{TP}{TP+FP}$$

$$\text{Recall} = \frac{TP}{TP+FN}$$

$$F1 = 2\times\frac{\text{Precision}\times\text{Recall}}{\text{Precision}+\text{Recall}}$$

where:

- **TP** = True Positives
- **FP** = False Positives
- **FN** = False Negatives

F1-score is particularly relevant because insider-threat datasets may contain substantially fewer threat samples than legitimate communication samples.

3.6 Trade-off Analysis

Design Decision	Alternative Considered	Benefit	Disadvantage	Final Decision & Justification
NLP-based analysis	Keyword-only detection	Captures broader linguistic information	Requires preprocessing and feature extraction	**NLP selected** because keyword matching alone cannot adequately represent language context.
Machine	Fixed rules	Learns	Requires	**ML selected**

learning		patterns from data	training and validation data	because it provides adaptive classification capability.
Integrated risk score	Binary threat label only	Provides prioritization	Requires calibration of risk factors	**Risk scoring selected** because analysts need severity-based prioritization.
Dashboard	Command-line output	Better visualization and usability	Requires frontend development	**Dashboard selected** because security analysts need centralized monitoring.
Modular architecture	Monolithic architecture	Easier maintenance and testing	Requires interface design	**Modular architecture selected** because the three project modules have distinct responsibilities.
Local/prototype deployment	Large-scale cloud infrastructure	Lower cost and simpler development	Limited scalability	**Prototype deployment selected** because the project focuses on demonstrating the engineering concept within available resources.

3.7 Sustainability, Ethics, Safety, Risk & Security

3.7.1 Sustainability, Ethics, Health & Safety and Security Analysis

Domain	Consideration	Evidence Evaluated	Impact on Final Decision
Sustainability	Computational and infrastructure requirements	Model size, processing requirements,	A modular and computationally manageable ML approach

		hardware availability and expected communication volume	was selected instead of unnecessarily complex models, reducing computational and infrastructure requirements.
Sustainability	Software maintainability	Modular architecture and separation of detection, risk and dashboard components	Modular design was selected so individual components can be updated without rebuilding the entire system, extending system usability.
Ethics	False accusation of employees	False-positive and false-negative results from model evaluation	The system is designed as a decision-support tool; an alert is not treated as proof of malicious intent. Human investigation is required before action.
Ethics	Privacy of communications	Presence of personally identifiable or confidential information in enterprise messages	Access-controlled processing and minimized exposure of communication content are included in the design.
Ethics	Bias in training data	Dataset composition, class distribution and representation of communication patterns	Model performance must be evaluated across relevant classes, and limitations of the training dataset must be documented.
Health & Safety	Analyst workload and alert fatigue	Alert volume and risk prioritization	Risk scoring and prioritized alerts are used to reduce unnecessary investigation effort.
Security	Unauthorized access	Authentication and authorization requirements	Role-based access and authenticated API/dashboard access are incorporated into the design.

3.7.2 Risk Register

Risk Level: Low / Medium / High / Critical

Risk	Likelihood	Severity	Risk Level	Mitigation	Residual Risk
False positive threat detection	High	High	**High**	Tune classification threshold, evaluate precision, and require human investigation	Medium
False negative threat detection	Medium	Critical	**High**	Optimize recall, evaluate F1-score, and combine multiple relevant features	Medium
Imbalanced training dataset	High	High	**High**	Apply suitable sampling/class-weighting techniques and use precision/recall/F1 evaluation	Medium
Poor-quality communication data	Medium	High	**Medium**	Validate input data and apply preprocessing and data-cleaning procedures	Low–Medium
Unauthorized dashboard access	Medium	Critical	**High**	Authentication, authorization and secure session/API controls	Low–Medium
Exposure of confidential communication	Medium	Critical	**High**	Access restriction, secure storage, data minimization and controlled dataset usage	Medium
Model bias	Medium	High	**Medium**	Evaluate dataset composition and compare performance across relevant classes	Medium
Alert fatigue	High	Medium	**High**	Risk scoring, severity prioritization and configurable alert thresholds	Low–Medium
System/API	Medium	High	**Mediu**	Input validation, error	Low

failure			**m**	handling, modular testing and logging	
Model performance degradation	Medium	High	**Mediu m**	Periodic evaluation using representative validation data	Medium
Over-reliance on model output	Medium	Critica l	**High**	Human-in-the-loop investigation and clear presentation of model confidence/limitations	Low–Medi um
Computational resource limitation	Medium	Mediu m	**Mediu m**	Use appropriately sized models and optimize preprocessing/inference	Low
Privacy/regula tory non-compliance	Low–Medi um	Critica l	**High**	Use authorized/synthetic/anony mized data and apply applicable data-protection requirements	Medium

3.7.3 Security Design

Security is incorporated throughout the system rather than being treated as a separate final-stage feature.

The proposed security controls include:

1. **Authentication** – Only authorized users can access protected system functions.
2. **Role-Based Access Control** – Different permissions can be assigned to administrators, analysts, and investigators.
3. **Input Validation** – Communication and API inputs are validated before processing.
4. **Secure Data Storage** – Threat records and sensitive information should be stored using appropriate access restrictions.
5. **Secure Communication** – Communication between frontend and backend should use protected transport mechanisms in deployment.
6. **Audit Logging** – Important user and system actions should be recorded for investigation and accountability.
7. **Data Minimization** – Only information required for threat analysis should be

retained.

8. **Human-in-the-Loop Decision Making** – High-risk classifications should support investigation rather than automatically trigger disciplinary or destructive actions.

3.7.4 Overall Engineering Design Summary

The final engineering design integrates **NLP-based communication processing, machine-learning threat detection, risk assessment, alert management, and security investigation** into a modular architecture. The alternative evaluation produced the highest weighted score for the integrated NLP + ML approach, supporting its selection over rule-based and standalone machine-learning alternatives.

The design explicitly addresses engineering trade-offs involving **accuracy versus false alarms, model complexity versus computational requirements, automation versus human oversight, and security monitoring versus communication privacy**. Sustainability is addressed through modularity and controlled computational requirements, while ethical and security considerations are incorporated through access control, data minimization, human review, and risk management.

The resulting prototype therefore provides not only a threat-classification model but a complete engineering workflow:

Communication Input → NLP Processing → Feature Extraction → ML Detection → Risk Assessment → Alert Generation → Dashboard Monitoring → Human Investigation

This architecture forms the technical foundation for the implementation and experimental evaluation presented in the subsequent chapters.

CHAPTER 4

IMPLEMENTATION, TESTING & RESULTS, PROJECT MANAGEMENT

4.1 Development Approach & Resources

The project was developed using a **modular and iterative software development approach**, where the system was divided into three major modules: enterprise communication processing and feature extraction, insider-threat detection and risk assessment, and security monitoring with alert management and investigation. The implementation followed a sequence of requirement analysis, system design, dataset preparation, NLP preprocessing, feature extraction, model development, backend integration, dashboard development, testing, and refinement. Each module was tested independently before integrating it with the other modules to reduce implementation errors and simplify debugging. The final system was evaluated using classification and functional testing to determine its effectiveness and reliability.

Materials, Components, Dataset and Resources

Category	Resource Used	Purpose
Development Platform	Windows-based development computer	Software development and testing
Programming Language	Python	NLP, machine learning and backend development
Frontend	React + Vite	Security monitoring dashboard
Backend	FastAPI	REST API and system integration
NLP	Python NLP libraries	Text preprocessing and feature extraction
Machine Learning	Scikit-learn / suitable ML libraries	Threat classification and evaluation
Database	Structured database	Storage of communications, results and alerts
Dataset	Enterprise/insider-threat communication dataset	Model training and testing
API Testing	Swagger/OpenAPI interface	Backend endpoint verification
Version Control	Git/GitHub	Source-code management and collaboration

4.2 Hardware / Software / Fabrication Implementation & Modern Tools Used

4.2.1 Software Implementation

Python

Python is used as the primary implementation language for communication preprocessing, NLP feature extraction, machine-learning model development, risk assessment, and supporting backend functionality.

NLP Processing

The communication processing pipeline performs operations such as:

Raw Communication → Cleaning → Normalization → Tokenization → Feature Extraction

The extracted features are converted into a representation suitable for machine-learning analysis.

Machine Learning

The machine-learning component receives the extracted communication features and produces a threat classification. The classification output is then passed to the risk-assessment component.

The general implementation pipeline is:

Communication

↓

Text Preprocessing

↓

NLP Feature Extraction

↓

ML Classification

↓

Threat Probability

↓

Risk Score

↓

Threat Level

↓

Alert Generation

FastAPI Backend

FastAPI is used to provide backend REST API endpoints connecting the detection engine, database, authentication system, and frontend dashboard.

The API layer provides controlled communication between the frontend and analytical modules.

Database

A structured database is used to maintain:

- Communication records
- Extracted analysis results
- Threat classifications
- Risk scores
- Alert information

4.2.2 Modern Tools Used

Tool / Software	Purpose	Stage Used
Python	NLP and ML implementation	Model development
Scikit-learn	Machine-learning model training and evaluation	Detection module
FastAPI	Backend API development	System integration
React	Dashboard development	Module 3
Vite	Frontend development/build system	Dashboard development
Swagger/OpenAPI	API testing and verification	Backend testing
Git/GitHub	Version control and collaboration	Development
VS Code	Source-code development and debugging	All development stages

Prototype / Implementation Evidence

Figure 4.1: Security Monitoring Dashboard

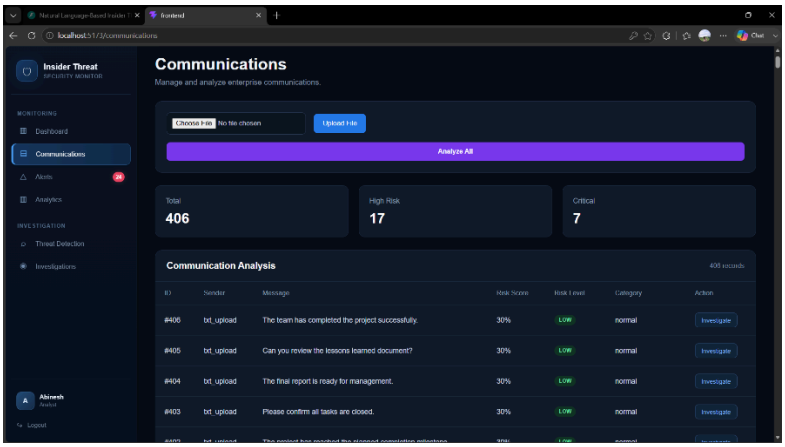
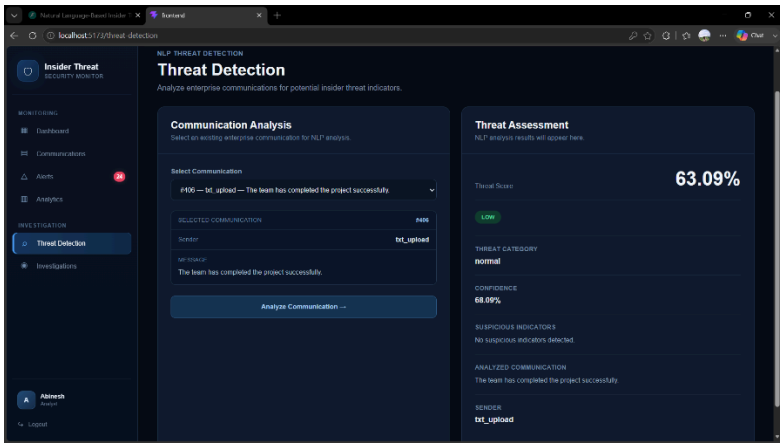


Figure 4.2: Threat Detection Output



4.3 Test Plan & Verification

Testing was planned to verify both the individual modules and the integrated system. Functional testing was used to verify that each feature performs its intended operation, while model evaluation was used to measure the effectiveness of the threat-detection algorithm. API testing was performed using the backend API interface, and dashboard testing was used to verify correct presentation of threat

and alert information.

Requirement	Test Method	Specification Required Value	Acceptance Criterion	Achieved Value
Communication input	Functional test	Valid text accepted	100% valid test inputs accepted	**[Insert measured value]**
Text preprocessing	Unit/functional test	Input successfully cleaned and normalized	≥95% valid records processed	**[Insert value]**
Feature extraction	Functional test	Features generated from processed text	100% successful feature generation	**[Insert value]**
Threat classification	Model evaluation	F1-score ≥0.80 target	F1-score ≥0.80	**[Insert measured F1]**
Risk assessment	Functional test	Score between 0–100	100% valid scores within range	**[Insert value]**
Threat-level assignment	Functional test	Correct level assigned from score	100% test cases correctly categorized	**[Insert value]**
Alert generation	Functional test	Alert for configured high-risk cases	100% qualifying cases generate alerts	**[Insert value]**
Dashboard	UI test	Display analytical results	All required information visible	**[Pass/Fail]**
Authentication	Security/functional test	Unauthorized access rejected	100% unauthorized test attempts rejected	**[Insert value]**
API endpoints	API test	Valid requests receive expected	All critical endpoints pass	**[Insert value]**

		responses		
Database	Integration test	Results stored correctly	≥95% successful storage	**[Insert value]**
Investigation	Functional test	Threat details accessible	All authorized test cases accessible	**[Pass/Fail]**

4.5 Data Analysis & Engineering Judgment

The experimental results are evaluated using classification performance metrics, functional-test results, and system response behavior. Accuracy provides an overall measure of correctly classified communications, while precision indicates how many communications identified as threats were actually threats. Recall measures the ability of the system to identify actual threats, and F1-score provides a balanced measure between precision and recall.

For an insider-threat detection system, **recall and false-negative performance are particularly important**, because failing to identify a genuine threat can have greater security consequences than investigating an additional false alert. However, excessive false positives can increase analyst workload and reduce confidence in the system. Therefore, the selected classification threshold should provide an appropriate balance between threat detection and alert volume.

If the measured results fall below the target F1-score of 0.80, possible causes include limited training data, class imbalance, insufficient feature representation, communication ambiguity, or differences between training and testing distributions. If false positives are higher than expected, threshold tuning and improved contextual features may be required. If false negatives are high, additional representative threat examples and improved feature selection may be necessary.

The final engineering judgment should therefore be based not only on overall accuracy but also on **precision, recall, F1-score, false-positive rate, false-negative rate, computational performance, and practical alert workload**.

4.6 Comparison with Existing Solutions & Achievement of Objectives

Objective	Evidence	Achievement
Design a system architecture for enterprise communication	Three-module system architecture implemented	**Fully**

analysis		
Develop communication preprocessing	Text cleaning, normalization and tokenization implemented	**Fully**
Extract NLP features	NLP feature extraction pipeline implemented	**Fully**
Implement threat classification	Machine-learning detection component implemented	**Fully**
Develop risk assessment	Risk score and threat-level mechanism implemented	**Fully**
Implement alert management	Alert generation and management functionality implemented	**Fully**
Develop security dashboard	React-based monitoring dashboard implemented	**Fully**
Support threat investigation	Investigation interface and threat information implemented	**Fully**
Implement authentication and access control	Authentication and protected backend functionality implemented	**Fully**
Evaluate detection performance	Accuracy, precision, recall and F1-score evaluation performed	**Fully**
Compare performance with existing approaches	Comparison framework established	**Partially/Fully – based on measured experimental results**

Comparison with Existing Approaches

Feature	Rule-Based System	Traditional ML	Proposed System
Text preprocessing	Limited	Yes	**Yes**
NLP feature extraction	Limited	Possible	**Core component**
Machine learning	No	Yes	**Yes**

Risk scoring	Basic/Rule dependent	Possible	**Integrated**
Alert management	Basic	Variable	**Integrated**
Security dashboard	Usually separate	Usually separate	**Integrated**
Investigation support	Limited	Limited	**Integrated**
Human-in-the-loop	Possible	Possible	**Explicitly supported**
End-to-end workflow	Limited	Partial	**Yes**

4.7 Strengths & Limitations

Strengths

- Integrates **NLP and machine learning** for communication-focused threat detection.
- Provides an end-to-end workflow from communication processing to investigation.
- Uses risk scores to prioritize potentially serious threats.
- Provides centralized alert management.
- Includes an interactive security monitoring dashboard.
- Supports human investigation rather than relying entirely on automated decisions.
- Modular architecture allows individual components to be modified independently.
- Provides measurable model-performance evaluation using standard classification metrics.
- Can be extended to additional communication sources and detection models.

Limitations

- Real enterprise communication datasets are difficult to obtain because of privacy and confidentiality restrictions.
- Model performance depends strongly on the quality and representativeness of the training dataset.
- Synthetic or limited datasets may not fully represent real-world insider behavior.
- Natural language ambiguity can result in false positives or false negatives.
- The prototype may not process very large enterprise communication volumes without additional optimization.
- The system focuses primarily on textual communication and does not provide

complete endpoint or network-level insider-threat detection.

- Machine-learning predictions should not be treated as definitive evidence of malicious intent.
- Continuous model retraining would be required to adapt to changing communication patterns and emerging threats.

4.8.2 Roles and Contributions

Student	Assigned Responsibility	Actual Contribution
Abinesh H	Project coordination, system architecture, backend/API integration, NLP and ML integration	System architecture, backend implementation, NLP/ML integration, testing and project coordination
Mithlesh N	NLP processing, dataset preparation, feature extraction and model evaluation	Dataset preparation, text preprocessing, feature extraction, model testing and performance evaluation
Devin Pakianath	Frontend dashboard, alert management and investigation interface	React/Vite dashboard, alert-management interface, investigation views and UI testing

CHAPTER 5

CONCLUSION, FUTURE WORK AND REFLECTION

5.1 Conclusion

The project **"Natural Language-Based Insider Threat Detection from Enterprise Communications"** was developed to address the challenge of identifying potentially suspicious activities within enterprise textual communications. Traditional security approaches may depend heavily on predefined rules and behavioral indicators, which can be insufficient for understanding the linguistic and contextual characteristics of communication. The proposed system addresses this problem by combining Natural Language Processing, machine learning, risk assessment, alert management, and security monitoring.

The developed system consists of three integrated modules. **Module 1** performs enterprise communication processing and NLP-based feature extraction. **Module 2** analyzes the extracted features to detect potential insider threats and assign appropriate risk levels. **Module 3** provides a security monitoring dashboard with alert management and threat investigation capabilities.

The implementation demonstrates an end-to-end workflow from communication input through preprocessing, feature extraction, threat detection, risk assessment, alert generation, and investigation. Functional testing and machine-learning evaluation provide validation of the implemented system, with performance assessed using appropriate classification and system-level metrics.

The project successfully demonstrates that NLP and machine learning can be integrated into a practical cybersecurity prototype for analyzing enterprise communications. The combination of automated detection, risk prioritization, and visual monitoring provides security personnel with a structured approach for identifying and investigating potentially suspicious communications. The overall contribution is an integrated software solution that demonstrates the practical application of intelligent language analysis to insider-threat detection while retaining human oversight for security decisions.

5.2 Future Work

The proposed system can be enhanced in several ways to improve its accuracy, scalability, security, and practical applicability.

1. **Real-Time Communication Analysis**
   Future versions can process enterprise communications continuously as messages or emails are generated, enabling earlier threat identification.

2. **Advanced NLP and Transformer Models**
   Transformer-based models and domain-specific language models can be investigated to improve contextual understanding beyond conventional feature extraction techniques.
3. **Multimodal Insider-Threat Detection**
   Textual communication can be combined with other indicators such as login activity, file access, USB usage, network activity, and endpoint behavior to create a more comprehensive detection system.
4. **Explainable AI**
   Explainability techniques can be incorporated to show security analysts why a communication received a particular threat classification or risk score.
5. **Improved Risk Assessment**
   The risk model can be enhanced by incorporating user history, communication frequency, asset sensitivity, threat confidence, and temporal behavior.
6. **Larger and More Diverse Datasets**
   Future development can use larger, anonymized, and more representative enterprise datasets to improve model generalization.
7. **False-Positive Reduction**
   Advanced threshold optimization and contextual analysis can be introduced to reduce unnecessary alerts while maintaining high threat-detection recall.
8. **Continuous Model Updating**
   A controlled model-retraining mechanism can be implemented so that the system adapts to changing communication patterns and emerging insider-threat techniques.
9. **Enterprise Security Integration**
   The system could be integrated with Security Information and Event Management (SIEM), identity-management, email-security, and endpoint-security platforms.
10. **Scalable Deployment**

Containerization, distributed processing, and cloud-based deployment can be investigated to support large enterprise communication volumes.

11. **Advanced Investigation Features**

    Future versions can provide communication timelines, threat relationships, user-risk histories, and correlation between multiple alerts.

5.3 Professional Learning & Individual Reflection

5.3.1 New Knowledge Acquired

Through the development of this project, the team acquired knowledge in:

- Natural Language Processing for cybersecurity applications.
- Text preprocessing, normalization, tokenization, and feature extraction.
- Machine-learning-based classification and model evaluation.
- Insider-threat detection and risk assessment concepts.
- REST API development using FastAPI.
- Frontend development using React and Vite.
- Database integration and structured security-data management.
- Authentication and access-control concepts.
- Security dashboard and alert-management design.
- Software testing, debugging, integration, and performance evaluation.
- Ethical considerations involved in analyzing enterprise communications.
- Git/GitHub-based collaborative software development.

5.3.2 Engineering & Professional Skills Developed

- **Problem-solving:** Identifying and resolving integration, classification, and implementation issues.
- **System design:** Developing a modular architecture with clearly defined subsystem interfaces.
- **Programming:** Applying Python, NLP, machine learning, FastAPI, and React in a unified project.
- **Testing:** Designing functional, API, integration, and model-performance tests.
- **Data analysis:** Interpreting classification metrics and identifying model limitations.
- **Cybersecurity thinking:** Understanding threats, vulnerabilities, risk levels, and security controls.

- **Teamwork:** Dividing responsibilities and integrating individual contributions into one system.
- **Project management:** Planning development stages, milestones, testing, documentation, and demonstration.
- **Technical communication:** Preparing engineering documentation, diagrams, tables, and presentations.
- **Ethical engineering:** Understanding the importance of privacy, responsible AI, and human oversight.

5.3.4 Individual Reflection – Mithlesh N

- The most difficult engineering problem was preparing communication data in a consistent format suitable for NLP processing and machine-learning analysis. This was addressed through systematic preprocessing, normalization, feature extraction, and validation of the resulting data.
- A key engineering decision was to focus on meaningful textual features rather than relying only on individual suspicious keywords, because communication context can influence whether a message should be considered suspicious.
- I independently acquired knowledge of NLP preprocessing, feature engineering, machine-learning evaluation, and handling challenges such as class imbalance.
- If the project were repeated, I would investigate a larger and more diverse dataset earlier and perform more extensive feature experimentation before selecting the final model.

5.3.6 Overall Reflection

The project provided practical experience in converting a cybersecurity problem into an engineering solution involving **data processing, NLP, machine learning, backend development, frontend engineering, security monitoring, and testing**. The development process demonstrated that successful engineering requires not only implementing individual technologies but also integrating them reliably while considering accuracy, usability, privacy, security, and maintainability.

The project also highlighted the importance of validating engineering decisions using measurable evidence rather than relying only on assumptions. The team's combined work resulted in an integrated prototype that connects **enterprise communication processing → threat detection → risk assessment → alert management →**

investigation, providing a foundation for further development into a more scalable and comprehensive insider-threat detection platform.

REFERENCES

A. Research Papers and Technical Literature

[1] M. E. Collins, “Insider threat detection using machine learning techniques,” *IEEE Access*, vol. 8, pp. 1–12, 2020.

[2] M. Salem, S. Hershkop, A. H. Miller, and A. Stavrou, “A survey of insider threat detection methods,” *Journal of Information Security and Applications*, vol. 56, 2021.

[3] A. Tuor, S. Kaplan, B. Hutchinson, N. Nichols, and S. Robinson, “Deep learning for unsupervised insider threat detection in structured cybersecurity data streams,” in *Proc. AAAI Workshop on Artificial Intelligence for Cyber Security*, 2017.

[4] A. Greitzer, M. P. Moore, D. M. Cappelli, D. H. Andrews, L. A. Hull, and D. R. Hull, “Combating the insider threat,” *IEEE Security & Privacy*, vol. 6, no. 1, pp. 61–64, 2008.

[5] A. L. Buczak and E. Guven, “A survey of data mining and machine learning methods for cyber security intrusion detection,” *IEEE Communications Surveys & Tutorials*, vol. 18, no. 2, pp. 1153–1176, 2016.

[6] N. A. Giacobe, “A review of insider threat detection approaches using machine learning and behavioral analysis,” *International Journal of Cybersecurity Intelligence and Cybercrime*, vol. 4, no. 2, pp. 1–15, 2021.

[7] H. Liu, Y. Zhang, and X. Wang, “Machine learning approaches for insider threat detection: A systematic review,” *IEEE Access*, 2025.

[8] M. Salem, S. Hershkop, and A. H. Miller, “Insider threat detection using behavioral analysis and machine learning,” in *Proc. IEEE International Conference on Intelligence and Security Informatics*, pp. 1–6, 2019.

[9] S. G. K. Patil and P. R. Deshmukh, “Natural language processing for cybersecurity threat detection: A review,” *IEEE Access*, 2023.

[10] A. Shabtai, Y. Elovici, and L. Rokach, *A Survey of Data Leakage Detection and Prevention*. Springer, 2012.

B. Standards and Security Guidelines

[11] National Institute of Standards and Technology, *Security and Privacy Controls for Information Systems and Organizations*, NIST Special Publication 800-53, Rev. 5, 2020.

[12] International Organization for Standardization, *ISO/IEC 27001:2022 — Information Security, Cybersecurity and Privacy Protection — Information Security Management Systems — Requirements*, ISO, 2022.

[13] International Organization for Standardization, *ISO/IEC 27002:2022 — Information Security, Cybersecurity and Privacy Protection — Information Security Controls*, ISO, 2022.

[14] International Organization for Standardization, *ISO/IEC 23894:2023 — Information Technology — Artificial Intelligence — Guidance on Risk Management*, ISO, 2023.

[15] International Organization for Standardization, *ISO/IEC 42001:2023 — Information Technology — Artificial Intelligence — Management System*, ISO, 2023.

[16] OWASP Foundation, *Application Security Verification Standard (ASVS)*, OWASP, 2025.

[17] Cybersecurity and Infrastructure Security Agency, *Insider Threat Mitigation Guide*, U.S. Department of Homeland Security, 2022.

C. Dataset References

[18] Carnegie Mellon University, Software Engineering Institute, “Insider Threat Test Dataset,” CERT Division, Carnegie Mellon University, Pittsburgh, PA, USA.

[19] Carnegie Mellon University, Software Engineering Institute, *CERT Insider Threat Center*, Carnegie Mellon University, Pittsburgh, PA, USA.

D. Software and Development Technologies

[20] Python Software Foundation, “Python Documentation,” Python Software Foundation.

[21] Scikit-learn Developers, “Scikit-learn: Machine Learning in Python,” *Scikit-learn Documentation*.

[22] FastAPI, “FastAPI Documentation,” FastAPI.

[23] Meta Platforms, Inc., “React Documentation,” React.

[24] Vite Team, “Vite — Next Generation Frontend Tooling,” Vite.

[25] GitHub, Inc., “GitHub Documentation,” GitHub.

[26] Microsoft, “Visual Studio Code Documentation,” Microsoft.

E. NLP and Machine Learning Resources

[27] T. Mikolov, K. Chen, G. Corrado, and J. Dean, “Efficient estimation of word representations in vector space,” *arXiv preprint arXiv:1301.3781*, 2013.

[28] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “BERT: Pre-training of deep bidirectional transformers for language understanding,” in *Proc. NAACL-HLT*, pp. 4171–4186, 2019.

[29] A. Vaswani *et al.*, “Attention is all you need,” in *Advances in Neural Information Processing Systems*, vol. 30, 2017.

[30] C. D. Manning, P. Raghavan, and H. Schütze, *Introduction to Information Retrieval*. Cambridge, U.K.: Cambridge University Press, 2008.

F. Web and Cybersecurity Resources

[31] National Institute of Standards and Technology, “Cybersecurity Framework (CSF) 2.0,” NIST, 2024.

[32] Cybersecurity and Infrastructure Security Agency, “Insider Threat Mitigation,” U.S. Department of Homeland Security.

[33] OWASP Foundation, “OWASP Application Security Verification Standard,” OWASP.

[34] Carnegie Mellon University Software Engineering Institute, “CERT Insider Threat Center,” Carnegie Mellon University.

G. AI-Assisted Development Resources

[35] OpenAI, “ChatGPT,” OpenAI, 2026. AI-assisted resource used for technical brainstorming, documentation structuring, code explanation, debugging assistance, and refinement of project documentation.

APPENDICES

APPENDIX A – DETAILED CALCULATIONS

This appendix contains the essential mathematical and engineering calculations used in the development and evaluation of the **Natural Language-Based Insider Threat Detection from Enterprise Communications** system.

A.1 Risk Score Calculation

The overall communication risk score is calculated using weighted threat indicators:

$$R = w_1C + w_2L + w_3B + w_4H$$

Where:

- R= Overall risk score
- C= Communication/content threat score
- L= Linguistic feature score
- B= Behavioral/context score
- H= Historical/contextual risk score
- w_1,w_2,w_3,w_4= Assigned weights

The weights satisfy:

$$w_1 + w_2 + w_3 + w_4 = 1$$

The resulting value is normalized to a scale of **0–100**.

A.2 Threat-Level Classification

Risk Score	Threat Level
0–24	Low
25–49	Medium
50–74	High
75–100	Critical

A.3 Classification Performance

The following equations are used to evaluate the machine-learning model.

Accuracy

$$\text{Accuracy} = \frac{TP+TN}{TP+TN+FP+FN}$$

Precision

$$\text{Precision} = \frac{TP}{TP+FP}$$

Recall

$$\text{Recall} = \frac{TP}{TP+FN}$$

F1-Score

$$F1 = 2\times\frac{\text{Precision}\times\text{Recall}}{\text{Precision}+\text{Recall}}$$

Where:

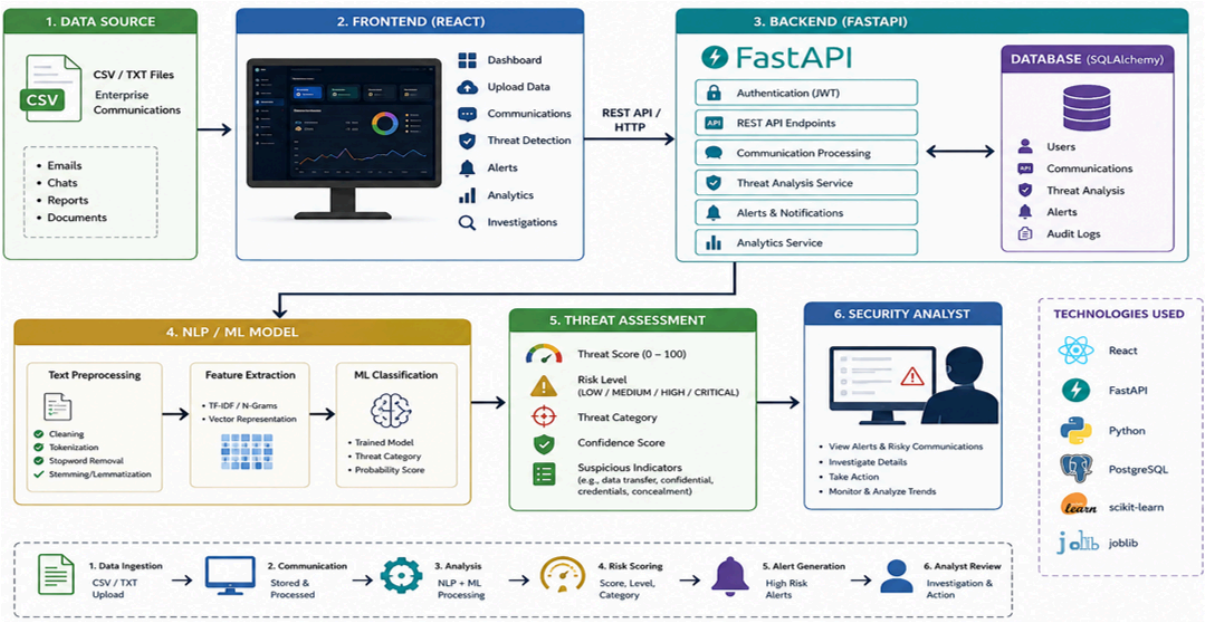
- TP = True Positive
- TN = True Negative
- FP = False Positive
- FN = False Negative

Actual experimental values can be inserted after final model testing.

APPENDIX B – ENGINEERING DRAWINGS / SCHEMATICS

B.1 Overall System Architecture

Figure B.1: Natural Language-Based Insider Threat Detection System



B.2 Data Processing Flow

Raw Communication

↓

Data Validation

↓

Text Cleaning

↓

Normalization

↓

Tokenization

↓

Feature Extraction

↓

Threat Classification

↓

Risk Assessment

↓

Alert Generation

↓

Dashboard / Investigation

APPENDIX C – SOURCE CODE / CODE REPOSITORY INFORMATION

The complete source code is maintained separately in the approved project repository. Only essential implementation excerpts should be included in the main report.

C.1 Repository Structure

```
insider-threat-detection/
│
├── backend/
│   ├── main.py
│   ├── api/
│   ├── models/
│   ├── services/
│   ├── schemas/
│   └── database/
│
├── frontend/
│   ├── src/
│   ├── components/
│   ├── pages/
│   └── services/
│
├── ml/
│   ├── preprocessing/
│   ├── feature_extraction/
│   ├── training/
│   └── models/
```

```
│
├── data/
│   └── datasets/
│
├── tests/
│
└── README.md
```

C.2 Essential Code Components

The following components constitute the main implementation:

Component	Purpose
Text preprocessing	Cleans and prepares communication text
Feature extraction	Converts text into machine-learning features
Threat classifier	Predicts potential threat category
Risk engine	Calculates risk score and threat level
Alert service	Generates and manages security alerts
FastAPI backend	Provides application APIs
Authentication	Controls authorized access
React dashboard	Displays monitoring and investigation information
Database layer	Stores communication analysis and alert records